CAPACITACIÓN QA AUTOMATION · PARTE 2 DE 2

Automatización web con Playwright, TypeScript y Cucumber

Cómo un escenario escrito en español abre un navegador real, actúa como un usuario y verifica lo que ve. Sin esperas mágicas, sin selectores frágiles.

Instructor **Giancarlo Palomino**

Node 24 · TypeScript · Playwright 1.64 · Cucumber 13

Octubre 2026

LO QUE TE LLEVAS HOY

Cuatro bloques, del navegador al escenario

1

Las piezas

Node, npm y TypeScript: qué hace cada uno y por qué los necesitamos.

2

Cómo piensa Playwright

Navegador, contexto y página. Localizadores, esperas automáticas y aserciones web-first.

3

Cucumber + Playwright

World, hooks y Page Objects: separar el qué del cómo.

4

Ejercicios

Cinco retos sobre SauceDemo. El último: encontrar los bugs que esconde un usuario "problemático".

REQUISITO

Entorno del **Manual de instalación · Parte 2** en verde: `npm test` → `3 scenarios (3 passed)`. Y haber visto la Parte 1: Gherkin funciona exactamente igual.

LA IDEA

Un robot que usa la web como una persona

Abre el navegador, escribe en los campos, hace clic y mira la pantalla. La diferencia: no se cansa, no se distrae y repite lo mismo cien veces igual.

Cuándo sí automatizar por pantalla

- Flujos críticos de punta a punta: login, compra, pago.
- Lo que se rompe seguido y cuesta revisar a mano.
- Lo que se repite en cada despliegue (regresión).

Cuándo no

- Reglas de negocio que se pueden probar por API (más rápido y estable: Parte 1).
- Pantallas que todavía cambian cada semana.
- Lo que se prueba una sola vez.

RECUERDA LA PIRÁMIDE

Pocas pruebas de UI, bien elegidas. Si diez escenarios de pantalla verifican lo mismo que una prueba de API, sobran nueve.

LA PILA

Cinco piezas, una encima de otra

Cucumber	El guion: lee los `.feature` y llama al código de cada paso.
Playwright	Las manos y los ojos: controla Chromium, Firefox y WebKit.
TypeScript	El idioma: JavaScript con tipos que detectan errores al escribir.
npm	La tienda: instala todo lo anterior en `node_modules`.
Node.js	El motor: ejecuta JavaScript fuera del navegador.

PACKAGE.JSON

La lista de compras del proyecto, como el `pom.xml` de la Parte 1.

TSX

El traductor que deja a Node leer `.ts` sin compilar a mano. Va en `NODE_OPTIONS`.

CROSS-ENV

Para que `HEADED=1` funcione igual en Windows, macOS y Linux.

COMPARADO CON LA PARTE 1

Node ↔ Java · npm ↔ Maven · Playwright ↔ REST Assured · Cucumber es el mismo, en otro idioma.

TYPESCRIPT EN UNA LÁMINA

El error aparece al escribir, no en la demo

JAVASCRIPT · FALLA AL EJECUTAR

login.steps.js ✗

```
async function ingresar(usuario, clave) { ... }

// Olvidé la clave: nadie avisa
await ingresar('standard_user');
// ...y 40 segundos después, la prueba falla
// con un mensaje que no menciona la causa.
```

TYPESCRIPT · FALLA AL ESCRIBIR

login.steps.ts ✓

```
async function ingresar(usuario: string, clave: string) {
... }

await ingresar('standard_user');
//    ~~~~~~~~ Expected 2 arguments, but got 1.
// VS Code lo subraya en rojo antes de ejecutar.
```

Tipos básicos

`string`, `number`, `boolean`, `string[]`.

async / await

Todo lo del navegador tarda: cada acción lleva `await`. Olvidarlo es el error número uno.

Autocompletado

Escribe `page.` y VS Code te muestra todo lo que puedes hacer.

TRES NIVELES

Navegador, contexto y página

Como muñecas rusas: un navegador contiene contextos, y cada contexto contiene páginas. El contexto es la clave del aislamiento.

BROWSER · SE ABRE UNA VEZ

Chromium

Caro de abrir (≈1 s). Lo compartimos entre todos los escenarios.

CONTEXT · UNO POR ESCENARIO

Escenario A

Cookies y sesión propias, como una ventana de incógnito.

PAGE la pestaña: `this.page`

CONTEXT · UNO POR ESCENARIO

Escenario B

No ve la sesión de A. Empieza de cero.

PAGE la pestaña: `this.page`

POR QUÉ IMPORTA

Si el escenario A inicia sesión y el B "hereda" esa sesión, B pasa cuando debería fallar. Un contexto nuevo por escenario (en el hook `Before`) elimina ese problema, y cuesta milisegundos.

ENCONTRAR ELEMENTOS

Un buen selector sobrevive a un rediseño

FRÁGIL · EVITAR

Atado a la estructura ✗

```
page.locator('#root > div > div:nth-child(2) > form > input')
page.locator('.btn.btn-primary.mt-3')
page.locator('xpath=//div[3]/button[1]')
```

Cambia un `div` o una clase de estilo y la prueba se rompe, aunque la app funcione.

ROBUSTO · PREFERIR

Atado al significado ✓

```
page.getByRole('button', { name: 'Login' })
page.getByLabel('Correo electrónico')
page.getByPlaceholder('Username')
page.locator('[data-test="login-button"]')
```

Describen qué es el elemento para el usuario, o usan atributos puestos para pruebas.

ORDEN DE PREFERENCIA

1. `getByRole` (como lo ve un lector de pantalla) · **2.** `getByLabel` / `getByPlaceholder` / `getByText` · **3.** `data-test` · **4.** CSS simple. XPath, solo si no queda otra. Para encontrarlos: *Pick locator* en VS Code o `npx playwright codegen`.

EL SUPERPODER DE PLAYWRIGHT

Esperas automáticas y aserciones web-first

ESPERA FIJA · NUNCA

Adivinar cuánto tarda ✗

```
await page.click('[data-test="login-button"]');
await page.waitForTimeout(3000);          // ¿y si tarda 4?
const titulo = await page.textContent('.title');
expect(titulo).toBe('Products');          // foto fija
```

Lenta cuando la app es rápida, rota cuando es lenta. La fuente nº 1 de pruebas "a veces fallan".

WEB-FIRST · SIEMPRE

Esperar a que se cumpla ✓

```
await page.locator('[data-test="login-button"]').click();
// click() ya esperó a que el botón exista,
// sea visible y esté habilitado.
await expect(page.locator('[data-test="title"]'))
  .toHaveText('Products');                // reintenta
hasta 5 s
```

Avanza en cuanto puede y solo falla si de verdad no ocurre.

VISIBILIDAD
`toBeVisible()`

TEXTO
`toHaveText()` · `toContainText()`

CANTIDAD
`toHaveCount(6)`

NAVEGACIÓN
`toHaveURL(/inventory/)`

03

Cucumber + Playwright

El mismo Gherkin de la Parte 1; ahora, cada paso mueve un navegador.

REPASO CON UN CASO WEB

El escenario no sabe que hay un navegador

features/login.feature — GHERKIN

```gherkin
# language: es
Característica: Inicio de sesión en SauceDemo
  Como comprador registrado
  Quiero iniciar sesión
  Para ver el catálogo de productos

  Antecedentes:
    Dado que estoy en la página de login

  @smoke
  Escenario: Login exitoso
    Cuando ingreso con el usuario "standard_user" y la clave "secret_sauce"
    Entonces veo el catálogo de productos

  @regresion
  Esquema del escenario: Login rechazado
    Cuando ingreso con el usuario "<usuario>" y la clave "<clave>"
    Entonces veo el mensaje de error "<mensaje>"
    Ejemplos:
      | usuario         | clave        | mensaje                                 |
      | locked_out_user | secret_sauce | Sorry, this user has been locked out |
```

PRUEBA DE FUEGO

Si mañana el login se hiciera por API, por app móvil o por voz, **este archivo no cambiaría**. Eso es un escenario declarativo.

LO QUE NO APARECE

Ni selectores, ni URLs, ni “hago clic en el botón azul”. Todo eso vive en el código.

EJEMPLOS QUE ENSEÑAN

Cada fila prueba una regla distinta: usuario bloqueado, clave errada. No repetir lo mismo con otros nombres.

EL VIAJE DE UNA FRASE

De “ingreso con el usuario…” a un clic real

1 · FEATURE
Gherkin
la frase en español

→

2 · STEP
When(…)
extrae usuario y clave

→

3 · WORLD
this.page
la pestaña de este escenario

→

4 · PAGE OBJECT
LoginPage
sabe dónde está cada campo

→

5 · PLAYWRIGHT
Navegador
escribe, hace clic, espera

Step definitions
Traducen. Cortos: reciben los datos de la frase y llaman al Page Object.

Page Objects
Conocen la pantalla: selectores y acciones. Una clase por página.

World + hooks
Preparan el navegador y entregan una página limpia a cada escenario.

UNA SOLA RESPONSABILIDAD POR CAPA

Si cambia el texto del escenario, tocas el step. Si cambia la pantalla, tocas el Page Object. Si cambia el ambiente, tocas el hook. Nunca las tres a la vez.

WORLD Y HOOKS

Qué pasa antes, durante y después de cada escenario

UNA VEZ
BeforeAll
chromium.launch()

→

CADA ESCENARIO
Before
newContext() + newPage()

→

CADA ESCENARIO
Steps
usan this.page

→

CADA ESCENARIO
After
captura si falló + cierra

→

UNA VEZ
AfterAll
browser.close()

src/support/hooks.ts · extracto — TYPESCRIPT

```typescript
Before(async function (this: CustomWorld) {
  this.context = await browser.newContext({ baseURL:
BASE_URL });
  this.page = await this.context.newPage();
});

After(async function (this: CustomWorld, { result }) {
  if (result?.status === Status.FAILED) {
    this.attach(await this.page.screenshot(), 'image/png');
  }
  await this.context?.close();
});
```

THIS
En los steps y hooks, `this` es el World del escenario. Por eso se usa `function`, no funciones flecha: la flecha pierde el `this`.

EVIDENCIA AUTOMÁTICA
Si un escenario falla, la captura queda adjunta en el reporte HTML. Nadie tiene que reproducirlo para ver qué pasó.

SETDEFAULTTIMEOUT(30_000)
Cucumber da 5 s por paso; abrir un navegador puede tardar más.

PAGE OBJECT

Los selectores viven en un solo lugar

SELECTORES EN LOS STEPS

login.steps.ts ✗

```ts
When('ingreso con el usuario {string} y la clave {string}',
  async function (this: CustomWorld, u: string, c: string) {
    await this.page.locator('[data-test="username"]').fill(u);
    await this.page.locator('[data-test="password"]').fill(c);
    await this.page.locator('[data-test="login-button"]').click();
  });
// ...y lo mismo copiado en checkout.steps.ts, carrito.steps.ts
```

CON PAGE OBJECT

login.steps.ts ✓

```ts
When('ingreso con el usuario {string} y la clave {string}',
  async function (this: CustomWorld, u: string, c: string) {
    await new LoginPage(this.page).ingresar(u, c);
  });

// LoginPage.ts es el único archivo que sabe
// que el campo se llama data-test="username".
```

LA PRUEBA DEL CAMBIO

Si desarrollo renombra `username` a `user-email`: sin Page Object cambias N archivos; con Page Object, una línea.

COMANDOS Y HERRAMIENTAS

Ejecutar, ver y depurar

COMANDO (WINDOWS, MACOS Y LINUX)	PARA QUÉ
`npm test`	Todo, sin ventana (CI)
`npm run test:smoke`	Solo `@smoke`
`npm run test:headed`	Ver el navegador
`npm test -- --tags @wip`	Lo que estás escribiendo
`npx playwright codegen saucedemo.com`	Grabar acciones y ver selectores
`npm run typecheck`	Revisar tipos sin ejecutar

PICK LOCATOR (VS CODE)

Con la extensión de Playwright, haz clic en un elemento de la página y obtén el mejor selector.

CODEGEN

Abre un navegador que escribe el código de lo que haces. Úsalo para descubrir selectores, no para copiar pruebas enteras.

REPORTS/CUCUMBER-REPORT.HTML

Cada escenario con sus pasos, duración y captura si falló.

AL DEPURAR

Primero `test:headed`: ver al robot equivocarse explica más que cualquier mensaje.

LEER EL ROJO

Cuatro errores que verás esta semana

`TimeoutError: locator.fill`

El selector no existe

Playwright esperó y el elemento nunca apareció. Revisa el selector con *Pick locator*.

`expect(…).toHaveText · Expected / Received`

La aserción falló

La página mostró otra cosa. Mira la captura adjunta en el reporte.

`Undefined. Implement with…`

La frase no tiene código

Una tilde o palabra distinta. Cucumber te da el snippet para pegar.

`function timed out … 5000 ms`

Paso demasiado lento

Falta `setDefaultTimeout`, o falta un `await` y el paso nunca termina.

EL ERROR SILENCIOSO

Olvidar `await` delante de una acción. El paso "pasa" al instante sin haber hecho nada, y el siguiente falla por una razón que parece no tener sentido.

CHECKLIST DEL BUEN ESCENARIO WEB

Seis reglas que revisaremos en cada entrega

1

Independiente

Pasa solo, en cualquier orden y en paralelo. Contexto nuevo por escenario.

2

Sin esperas fijas

Cero `waitForTimeout`. Aserciones web-first.

3

Selectores con significado

`getByRole` o `data-test`. Nada de `nth-child`.

4

Declarativo

El feature habla de negocio; el cómo vive en el código.

5

Una sola fuente

Cada selector existe en un único Page Object.

6

Verifica algo útil

Todo escenario termina con un *Entonces* que puede fallar.

04

Ejercicios

Cinco retos sobre `saucedemo.com`, una tienda de práctica. Todos parten del proyecto base. El quinto: encontrar bugs reales.

01
Cerrar sesión.

02
Carrito guiado por datos.

03 · 04
Ordenamiento real y compra de punta a punta.

05
Caza de bugs con `problem_user`.

1
NIVEL

CALENTAMIENTO

Cerrar sesión y no poder volver atrás

25 min

Dificultad

OBJETIVO

Escribir el escenario de cierre de sesión con un Page Object nuevo para el menú lateral.

CRITERIOS DE ACEPTACIÓN

1. Tras iniciar sesión, abrir el menú y elegir *Logout* lleva de vuelta a la pantalla de login.
2. Si después se navega directo a `/inventory.html`, la app no muestra productos: muestra un error.
3. Nuevo `MenuPage.ts`; los steps no tienen selectores.
4. Reutiliza el paso de login existente: no lo dupliques.

Punto de partida — GHERKIN

```gherkin
@ejercicio1
Escenario: Cerrar sesión
  Cuando ingreso con el usuario "standard_user" y la clave
"secret_sauce"
  Y cierro sesión desde el menú
  Entonces vuelvo a la página de login

@ejercicio1
Escenario: Sin sesión no se ve el catálogo
  Cuando intento abrir el catálogo sin iniciar sesión
  Entonces veo el mensaje de error "You can only access"
```

PISTAS

Botón del menú: `#react-burger-menu-btn` · opción: `[data-test="logout-sidebar-link"]` · `await expect(this.page).toHaveURL('https://www.saucedemo.com/')`.

Aprendes: crear un Page Object, `toHaveURL`, reutilizar pasos, probar el control de acceso.

2
NIVEL

DATOS

Un carrito que cuenta bien

35 min

Dificultad

OBJETIVO

Agregar varios productos desde una lista de Gherkin y verificar el contador del carrito y su contenido.

CRITERIOS DE ACEPTACIÓN

1. El paso recibe una lista de productos (tabla de una columna) y los agrega uno por uno.
2. El contador del carrito muestra exactamente la cantidad agregada.
3. En el carrito aparecen esos nombres, ni uno más.
4. Quitar un producto baja el contador en 1; quitar todos hace desaparecer el contador.

Aprendes: Data Tables en TypeScript (`DataTable.raw()`), `toHaveCount`, `allTextContents`, verificar que algo **no** existe con `not.toBeVisible()`.

Punto de partida — GHERKIN

```
@ejercicio2
Escenario: El carrito refleja lo que agrego
  Dado que inicié sesión como "standard_user"
  Cuando agrego al carrito los productos:
    | Sauce Labs Backpack     |
    | Sauce Labs Bike Light   |
    | Sauce Labs Bolt T-Shirt |
  Entonces el carrito indica 3 productos
  Y el carrito contiene exactamente esos productos
  Cuando quito "Sauce Labs Bike Light" del carrito
  Entonces el carrito indica 2 productos
```

PISTAS

Contador: `[data-test="shopping-cart-badge"]` · botón por nombre: `page.locator('[data-test="inventory-item"]', { hasText: nombre }).getByRole('button', { name: 'Add to cart' })` — así no dependes del id del producto.

3
NIVEL

VERIFICAR DE VERDAD

¿El ordenamiento ordena?

40 min
Dificultad

OBJETIVO

No basta con elegir "precio de menor a mayor" y ver que la página no falló. Hay que **leer todos los precios y comprobar el orden**.

CRITERIOS DE ACEPTACIÓN

1. Un *Esquema* cubre las 4 opciones: nombre A→Z, Z→A, precio menor→mayor, mayor→menor.
2. El step lee los valores de la página, los convierte (`"$7.99"` → `7.99`) y los compara con una copia ordenada por el propio código.
3. Si falla, el mensaje muestra la lista obtenida y la esperada.
4. Ningún valor esperado está escrito a mano en el feature.

Aprendes: `selectOption`, leer listas con `allTextContents()`, transformar datos, aserciones sobre colecciones. Es la técnica que en el ejercicio 5 encuentra un bug.

Punto de partida — GHERKIN + TS

```
@ejercicio3
Esquema del escenario: Ordenar el catálogo
  Dado que inicié sesión como "standard_user"
  Cuando ordeno los productos por "<criterio>"
  Entonces los productos quedan ordenados por "<criterio>"
  Ejemplos:
    | criterio |
    | az       |
    | za       |
    | lohi     |
    | hilo     |

const precios = (await page.locator('[data-test="inventory-item-price"]')
  .allTextContents()).map((p) => Number(p.replace('$', '')));
expect(precios).toEqual([...precios].sort((a, b) => a - b));
```

PISTA

Selector del orden: `[data-test="product-sort-container"]` (con guiones; tutoriales viejos lo escriben con guion bajo y ya no funciona).

4
NIVEL

DE PUNTA A PUNTA

Una compra completa que cuadra

50 min

Dificultad ●●●●○

OBJETIVO

Automatizar el flujo crítico de la tienda: catálogo → carrito → datos de envío → resumen → confirmación, verificando que **los montos cuadran**.

CRITERIOS DE ACEPTACIÓN

1. Un Page Object por pantalla: `InventoryPage`, `CartPage`, `CheckoutPage`.
2. Los datos del comprador llegan por Data Table (nombre, apellido, código postal).
3. En el resumen: subtotal = suma de precios; impuesto = 8 % del subtotal; total = subtotal + impuesto (redondeo a 2 decimales).
4. La confirmación muestra *Thank you for your order!*
5. Sin datos de envío, la app muestra un error y no avanza (escenario negativo).

Punto de partida — GHERKIN

```
@ejercicio4 @smoke
Escenario: Comprar dos productos
  Dado que inicié sesión como "standard_user"
  Y que agregué al carrito "Sauce Labs Backpack" y "Sauce Labs Bike
Light"
  Cuando completo la compra con los datos:
    | nombre       | Ana    |
    | apellido     | Torres |
    | codigoPostal | 15001  |
  Entonces el resumen muestra subtotal, impuesto y total correctos
  Y veo la confirmación "Thank you for your order!"
```

DATO VERIFICADO

Mochila ($29.99) + luz ($9.99): subtotal **$39.98**, impuesto **$3.20**, total **$43.18**. Selectores: `subtotal-label`, `tax-label`, `total-label`, `complete-header` (todos `data-test`).

Aprendes: varios Page Objects colaborando, cálculos en aserciones, flujos E2E y su escenario negativo.

5
RETO

PENSAR COMO QA

Caza de bugs: el usuario problemático

60 min
Dificultad ●●●●●

OBJETIVO

SauceDemo tiene usuarios con fallos a propósito: `problem_user`, `error_user`, `visual_user` y `performance_glitch_user` (clave `secret_sauce`). Ejecuta tus escenarios de los ejercicios 2 a 4 con ellos y encuentra lo que se rompe.

PISTAS, SIN SPOILERS

1. Mira con atención las imágenes del catálogo de `problem_user`.
2. Tu ejercicio 3 ya sabe si el ordenamiento de verdad ordena.
3. En el checkout, ¿lo que escribes queda en el campo donde lo escribiste?
4. ¿Cuánto tarda en entrar `performance_glitch_user`? ¿Tus pruebas lo toleran?

Aprendes: parametrizar el usuario con *Esquemas*, que una buena aserción encuentra bugs sin buscarlos, redactar reportes y etiquetar con `@bug`.

Y PARA SUBIR DE NIVEL

Varios navegadores

Una variable `BROWSER` (chromium, firefox, webkit) que el hook lea para lanzar el navegador elegido.

En paralelo

`parallel: 3` en `cucumber.js`. Si algo falla solo en paralelo, tus escenarios no eran independientes.

Red simulada

Con `page.route`, bloquea las imágenes y verifica que la página siga siendo usable.

ENTREGABLE

Escenarios `@bug` + `BUGS.md` con título, usuario, pasos, esperado, obtenido, captura y severidad.

CÓMO SE REVISA

Entrega y rúbrica

CRITERIO	PESO	QUÉ SE MIRA
Funciona	40 %	`npm test` verde (salvo `@bug` documentados) y `npm run typecheck` sin errores.
Escenarios	25 %	Declarativos, independientes, verifican algo que puede fallar.
Código	20 %	Page Objects, cero `waitForTimeout`, selectores con significado, sin duplicar steps.
Criterio QA	15 %	Casos negativos, calidad del `BUGS.md` y las capturas.

CÓMO ENTREGAR

Una rama por ejercicio, un PR con el reporte HTML adjunto y commits en español: `feat: agrega escenario de checkout`.

LO QUE MÁS RESTA

Un `waitForTimeout` "para que pase". Si necesitas esperar, falta una aserción web-first.

CANVIA

PARA LLEVARTE

Lo mismo que en APIs, ahora con un navegador

1 · El escenario no cambia
Gherkin describe el negocio; Playwright o REST Assured son solo el cómo.

2 · Esperar bien
Aserciones web-first y cero esperas fijas: pruebas rápidas que no mienten.

3 · Cada cosa en su capa
Steps que traducen, Page Objects que conocen la pantalla, hooks que preparan.

Documentación: **playwright.dev** · **cucumber.io/docs**

Práctica: **saucedemo.com** · **restful-booker.herokuapp.com**